

Backend Schema Document

Take Your Trip (tytluxe.in) — Database, Auth, Permissions & Data Ownership

Version: 1.0 (Draft) **Owner:** [Your name], Senior Backend Engineer **Companion docs:** PRD, TRD, App Flow Document, UI/UX Design Brief **Stack assumed:** Laravel 12, MySQL 8, Filament 3, Redis (sessions/queue/cache) **Purpose:** Canonical schema for implementation — every table, column, type, key, index, relationship, and access rule needed to build without re-deriving decisions already made in the TRD.

0. Conventions

- All tables use `id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY` (Laravel default `id()`).
 - All tables include `created_at`, `updated_at` (`TIMESTAMP NULL`, Laravel `timestamps()`), omitted from column lists below unless noted otherwise — assume present.
 - Soft-deletable tables include `deleted_at TIMESTAMP NULL` (Laravel `softDeletes()`) — flagged per table.
 - Foreign keys use `ON DELETE` behavior explicitly noted per relationship — never left to default.
 - Enum columns are implemented as MySQL `ENUM` or a `VARCHAR` + Laravel PHP `enum` cast — either is acceptable; `ENUM` type shown here for clarity of allowed values.
 - Money columns: `DECIMAL(10,2)` — never `FLOAT`/`DOUBLE` for currency.
 - All `slug` columns: `VARCHAR(191) UNIQUE`, generated from title on save.
-

1. Entity Relationship Overview (textual)

users —< enquiries (nullable FK, guest allowed)
users —< bookings (nullable FK, guest allowed)
users —< reviews
users >—< roles (via role_user, or a single `role` column – see §5)

destinations —< hotels
destinations —< staycations (optional FK)

hotels —< hotel_images
hotels —< room_types
hotels —< hotel_amenity (pivot) >— amenities
hotels —< enquiries (polymorphic reference)
hotels —< bookings (polymorphic reference)
hotels —< reviews (polymorphic reference)

cruises —< cruise_itinerary_days
cruises —< cruise_cabin_types
cruises —< enquiries (polymorphic reference)

staycations —< enquiries (polymorphic reference)
packages —< package_inclusions
packages —< enquiries (polymorphic reference)

offers — (optional FK to any vertical listing, + vertical enum)

bookings —< booking_travelers
bookings —< payments
bookings }— room_types (nullable FK, hotel bookings only)

enquiries }— users (assigned_agent_id FK)

admin_activity_logs }— users (actor)

2. Core Tables

2.1 `users`

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
name	VARCHAR(191)	NOT NULL
email	VARCHAR(191)	NOT NULL, UNIQUE
phone	VARCHAR(20)	NULL, UNIQUE (nullable unique — allows multiple NULLs)
password	VARCHAR(255)	NOT NULL (hashed)
role	ENUM('customer','agent','admin')	NOT NULL, DEFAULT 'customer'
email_verified_at	TIMESTAMP	NULL
phone_verified_at	TIMESTAMP	NULL
is_active	BOOLEAN	NOT NULL, DEFAULT true — used to deactivate agent/admin accounts without deleting
remember_token	VARCHAR(100)	NULL (Laravel "remember me")
two_factor_secret	TEXT	NULL, encrypted (agents/admins only, per TRD §5)
two_factor_recovery_codes	TEXT	NULL, encrypted
deleted_at	TIMESTAMP	NULL (soft delete — never hard-delete a user with booking/payment history)

Indexes: UNIQUE(email), UNIQUE(phone), INDEX(role).

Notes:

- Single `role` column (not a full RBAC package) is sufficient for 3 fixed roles per TRD scope — avoids Spatie Permission overhead unless granular per-resource permissions are needed later (see §5.3 for how Filament policies still enforce fine-grained rules on top of this simple role column).
- `phone` nullable+unique: customers registering via email-only flows shouldn't be blocked; phone can be added later in profile.

2.2 destinations

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
name	VARCHAR(191)	NOT NULL
slug	VARCHAR(191)	UNIQUE, NOT NULL
country	VARCHAR(100)	NOT NULL
type	ENUM('city','region','island','other')	NOT NULL, DEFAULT 'city'
lat	DECIMAL(10,7)	NULL
lng	DECIMAL(10,7)	NULL
is_active	BOOLEAN	NOT NULL, DEFAULT true

Indexes: UNIQUE(slug), INDEX(country), FULLTEXT(name) for autocomplete search.

Used by: Hotels (FK), Staycations (optional FK), Flight search autocomplete (read-only reference, flights themselves aren't stored — TRD §4.2).

3. Hotels Module

3.1 hotels

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
destination_id	BIGINT UNSIGNED	FK → destinations.id, ON DELETE RESTRICT
title	VARCHAR(191)	NOT NULL
slug	VARCHAR(191)	UNIQUE, NOT NULL
description	TEXT	NOT NULL
category	ENUM('beach_resort','city_luxury','honeymoon','family_friendly')	NOT NULL
address	VARCHAR(255)	NOT NULL
lat	DECIMAL(10,7)	NULL
lng	DECIMAL(10,7)	NULL
star_rating	TINYINT UNSIGNED	NOT NULL, CHECK (1-5)
price_from	DECIMAL(10,2)	NOT NULL
source	ENUM('tripjack','manual')	NOT NULL, DEFAULT 'manual'
tripjack_hotel_id	VARCHAR(100)	NULL, UNIQUE (nullable unique) — required if source = tripjack
is_active	BOOLEAN	NOT NULL, DEFAULT true
is_featured	BOOLEAN	NOT NULL, DEFAULT false (drives

Column	Type	Constraints
		homepage "curated picks")
deleted_at	TIMESTAMP	NULL (soft delete — bookings reference hotels historically)

Indexes: UNIQUE(slug), UNIQUE(tripjack_hotel_id), INDEX(destination_id), INDEX(category), INDEX(is_active), FULLTEXT(title, description).

3.2 hotel_images

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
hotel_id	BIGINT UNSIGNED	FK → hotels.id, ON DELETE CASCADE
path	VARCHAR(255)	NOT NULL (S3/storage path per TRD §7.2)
sort_order	SMALLINT UNSIGNED	NOT NULL, DEFAULT 0
alt_text	VARCHAR(191)	NULL (accessibility, per Design Brief §11)

Indexes: INDEX(hotel_id, sort_order).

3.3 **room_types**

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
hotel_id	BIGINT UNSIGNED	FK → hotels.id, ON DELETE CASCADE
name	VARCHAR(191)	NOT NULL
occupancy_adults	TINYINT UNSIGNED	NOT NULL
occupancy_children	TINYINT UNSIGNED	NOT NULL, DEFAULT 0
price_per_night	DECIMAL(10,2)	NOT NULL (base/reference price for manual hotels; TripJack hotels get live price at search time, this is a fallback display value)
cancellation_policy	ENUM('free_cancellation','non_refundable','partial')	NOT NULL
tripjack_room_code	VARCHAR(100)	NULL
is_active	BOOLEAN	NOT NULL, DEFAULT true

Indexes: INDEX(hotel_id).

3.4 **amenities**

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
name	VARCHAR(100)	NOT NULL, UNIQUE
icon	VARCHAR(100)	NULL (icon class/key per Design Brief §6)

3.5 **amenity_hotel** (pivot)

Column	Type	Constraints
amenity_id	BIGINT UNSIGNED	FK → amenities.id, ON DELETE CASCADE
hotel_id	BIGINT UNSIGNED	FK → hotels.id, ON DELETE CASCADE

Indexes: PRIMARY KEY(amenity_id, hotel_id).

4. Cruises Module (curated only, per TRD §4.2 — no live supplier)

4.1 cruises

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
title	VARCHAR(191)	NOT NULL
slug	VARCHAR(191)	UNIQUE, NOT NULL
description	TEXT	NOT NULL
cruise_line	VARCHAR(191)	NOT NULL
category	ENUM('scenic_getaway','luxury','international')	NOT NULL
duration_nights	SMALLINT UNSIGNED	NOT NULL
price_from	DECIMAL(10,2)	NOT NULL
is_active	BOOLEAN	NOT NULL, DEFAULT true
deleted_at	TIMESTAMP	NULL (soft delete)

Indexes: UNIQUE(slug), INDEX(category), FULLTEXT(title, description).

4.2 cruise_images

Same shape as hotel_images (§3.2), FK (cruise_id → cruises.id ON DELETE CASCADE).

4.3 cruise_itinerary_days

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
cruise_id	BIGINT UNSIGNED	FK → cruises.id, ON DELETE CASCADE
day_number	SMALLINT UNSIGNED	NOT NULL
port_name	VARCHAR(191)	NOT NULL
description	TEXT	NULL

Indexes: INDEX(cruise_id, day_number).

4.4 `cruise_cabin_types`

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
cruise_id	BIGINT UNSIGNED	FK → cruises.id, ON DELETE CASCADE
name	VARCHAR(191)	NOT NULL (e.g., "Ocean View Balcony")
description	TEXT	NULL
price_from	DECIMAL(10,2)	NULL

Used by: the Enquiry form's cabin-type dropdown context (App Flow §7.2).

5. Staycations Module

5.1 `staycations`

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
destination_id	BIGINT UNSIGNED	FK → destinations.id, ON DELETE SET NULL, NULL
title	VARCHAR(191)	NOT NULL
slug	VARCHAR(191)	UNIQUE, NOT NULL
description	TEXT	NOT NULL
duration_nights	SMALLINT UNSIGNED	NOT NULL
price_from	DECIMAL(10,2)	NOT NULL
is_active	BOOLEAN	NOT NULL, DEFAULT true
deleted_at	TIMESTAMP	NULL

Indexes: UNIQUE(slug), INDEX(destination_id), FULLTEXT(title, description).

5.2 `staycation_amenities` (pivot, reuses `amenities` table from §3.4)

Column	Type	Constraints
amenity_id	BIGINT UNSIGNED	FK → amenities.id, ON DELETE CASCADE
staycation_id	BIGINT UNSIGNED	FK → staycations.id, ON DELETE CASCADE

Indexes: PRIMARY KEY(amenity_id, staycation_id).

5.3 staycation_images

Same shape as §3.2, FK to staycations.id.

6. Packages Module

6.1 packages

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
title	VARCHAR(191)	NOT NULL
slug	VARCHAR(191)	UNIQUE, NOT NULL
description	TEXT	NOT NULL
duration_nights	SMALLINT UNSIGNED	NOT NULL
price_from	DECIMAL(10,2)	NOT NULL
is_active	BOOLEAN	NOT NULL, DEFAULT true
deleted_at	TIMESTAMP	NULL

Indexes: UNIQUE(slug), FULLTEXT(title, description).

6.2 package_inclusions

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
package_id	BIGINT UNSIGNED	FK → packages.id, ON DELETE CASCADE
label	VARCHAR(191)	NOT NULL (e.g., "Return Flights," "4-star Hotel," "Airport Transfers")
category	ENUM('flight','hotel','meal','activity','transfer','other')	NOT NULL
sort_order	SMALLINT UNSIGNED	NOT NULL, DEFAULT 0

6.3 package_images

Same shape as §3.2, FK to packages.id.

7. Offers

7.1 offers

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
title	VARCHAR(191)	NOT NULL
description	TEXT	NOT NULL
discount_type	ENUM('percentage','fixed')	NOT NULL
discount_value	DECIMAL(8,2)	NOT NULL
promo_code	VARCHAR(50)	NULL, UNIQUE
applies_to_vertical	ENUM('hotel','flight','cruise','staycation','package','all')	NOT NULL
applies_to_reference_id	BIGINT UNSIGNED	NULL (optional FK to a specific listing — polymorphic-lite, see §12.1 pattern; NULL = applies broadly to the vertical)
valid_from	DATE	NOT NULL
valid_to	DATE	NOT NULL
is_active	BOOLEAN	NOT NULL, DEFAULT true (auto-managed by scheduled job per TRD §3.4/ §7.3, manually overridable)

Indexes: UNIQUE(promo_code), INDEX(applies_to_vertical), INDEX(valid_from, valid_to), INDEX(is_active).

Validation rule (app-layer, not DB): `valid_to` must be after `valid_from` (enforced in Filament form + model observer, per App Flow §15.3).

8. Enquiries (shared across all verticals — TRD §4.2 design decision)

8.1 enquiries

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
user_id	BIGINT UNSIGNED	FK → users.id, ON DELETE SET NULL, NULL (guest-submittable)
vertical	ENUM('hotel','flight','cruise','staycation','package','general')	NOT NULL
reference_id	BIGINT UNSIGNED	NULL (points to hotels.id / cruises.id / staycations.id / packages.id depending on vertical — app-layer polymorphic, see §12.1)
name	VARCHAR(191)	NOT NULL
phone	VARCHAR(20)	NOT NULL
email	VARCHAR(191)	NOT NULL
travel_date_from	DATE	NULL
travel_date_to	DATE	NULL
pax_adults	TINYINT UNSIGNED	NOT NULL, DEFAULT 1
pax_children	TINYINT UNSIGNED	NOT NULL, DEFAULT 0
notes	VARCHAR(500)	NULL
status	ENUM('new','contacted','quoted','converted','closed')	NOT NULL, DEFAULT 'new'
assigned_agent_id	BIGINT UNSIGNED	FK → users.id, ON DELETE SET

Column	Type	Constraints
		NULL, NULL
source	ENUM('web','whatsapp','phone')	NOT NULL, DEFAULT 'web'

Indexes: INDEX(vertical, reference_id), INDEX(status), INDEX(assigned_agent_id), INDEX(user_id), INDEX(email), INDEX(phone), INDEX(created_at) — for admin filtering/sorting per App Flow §15.4.

Rate-limiting support: app-layer check on ((phone OR email, created_at)) before insert, per App Flow §11's soft-block rule — not a DB constraint, enforced in the EnquiryController/Form Request.

9. Bookings & Payments (Hotels + Flights only in Phase 1, per TRD §0

phasing)

9.1 bookings

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
reference	VARCHAR(20)	UNIQUE, NOT NULL (human-readable booking ref, e.g., <u>TYT-2026-000123</u> , generated on create)
user_id	BIGINT UNSIGNED	FK → users.id, ON DELETE SET NULL, NULL (guest bookings allowed)
guest_email	VARCHAR(191)	NULL (required if user_id is NULL — app- layer rule, used for guest confirmation lookup)
guest_phone	VARCHAR(20)	NULL
vertical	ENUM('hotel','flight')	NOT NULL
hotel_id	BIGINT UNSIGNED	FK → hotels.id, ON DELETE RESTRICT, NULL (set if vertical=hotel)
room_type_id	BIGINT UNSIGNED	FK → room_types.id, ON DELETE RESTRICT, NULL (set if vertical=hotel)
tripjack_booking_id	VARCHAR(100)	NULL, UNIQUE (nullable unique)
tripjack_hold_id	VARCHAR(100)	NULL (transient hold reference before confirm, TRD §6.1)
check_in	DATE	NULL (hotel only)
check_out	DATE	NULL (hotel only)

Column	Type	Constraints
flight_route	VARCHAR(255)	NULL (flight only — origin/destination/date summary; full itinerary in <code>traveler_details</code> JSON or a separate <code>booking_flight_legs</code> table if multi-city support is built, see §9.1a)
pax_adults	TINYINT UNSIGNED	NOT NULL, DEFAULT 1
pax_children	TINYINT UNSIGNED	NOT NULL, DEFAULT 0
lead_guest_name	VARCHAR(191)	NOT NULL
special_requests	VARCHAR(500)	NULL
base_amount	DECIMAL(10,2)	NOT NULL
tax_amount	DECIMAL(10,2)	NOT NULL, DEFAULT 0
discount_amount	DECIMAL(10,2)	NOT NULL, DEFAULT 0
total_amount	DECIMAL(10,2)	NOT NULL
currency	CHAR(3)	NOT NULL, DEFAULT 'INR'
offer_id	BIGINT UNSIGNED	FK → offers.id, ON DELETE SET NULL, NULL
status	ENUM('pending_payment','confirmed','cancelled','failed_needs_review')	NOT NULL, DEFAULT 'pending_payment'
cancellation_reason	VARCHAR(255)	NULL
admin_note	TEXT	NULL (manual override audit trail, App Flow §15.5)

Indexes: UNIQUE(reference), UNIQUE(tripjack_booking_id), INDEX(user_id), INDEX(status), INDEX(vertical), INDEX(guest_email), INDEX(created_at).

Note on flight legs: if multi-city flights need full structured storage beyond a summary string, add a `booking_flight_legs` child table (`booking_id` FK), `leg_number`, `origin`,

(`destination`), (`departure_at`), (`arrival_at`), (`airline`), (`flight_number`), (`fare_class`)) — included here as a documented extension point rather than pre-built, since the App Flow (§17) already flags multi-city rules as needing TripJack-specific confirmation first.

9.2 `booking_travelers`

(Per-passenger detail — required for flights per App Flow §6.3; optional/single-row for hotels where only lead guest matters.)

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
booking_id	BIGINT UNSIGNED	FK → bookings.id, ON DELETE CASCADE
full_name	VARCHAR(191)	NOT NULL
date_of_birth	DATE	NULL
gender	ENUM('male','female','other','prefer_not_to_say')	NULL
passport_number	VARCHAR(50)	NULL (encrypted at rest, international flights only)
passport_expiry	DATE	NULL
traveler_type	ENUM('adult','child','infant')	NOT NULL, DEFAULT 'adult'

Indexes: INDEX(booking_id).

9.3 payments

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
booking_id	BIGINT UNSIGNED	FK → bookings.id, ON DELETE CASCADE
razorpay_order_id	VARCHAR(100)	NOT NULL, UNIQUE
razorpay_payment_id	VARCHAR(100)	NULL, UNIQUE (nullable unique — set once payment completes)
razorpay_signature	VARCHAR(255)	NULL
amount	DECIMAL(10,2)	NOT NULL
currency	CHAR(3)	NOT NULL, DEFAULT 'INR'
status	ENUM('created','authorized','captured','failed','refunded','partially_refunded')	NOT NULL, DEFAULT 'created'
method	VARCHAR(50)	NULL (card/upi/netbanki from Razorpay response)
raw_response	JSON	NULL (full webhook payload, for audit — TRD §6.2/§8)
refund_amount	DECIMAL(10,2)	NULL
refund_reason	VARCHAR(255)	NULL

Indexes: UNIQUE(razorpay_order_id), UNIQUE(razorpay_payment_id), INDEX(booking_id), INDEX(status).

Immutability rule (app-layer): rows in payments are never updated by anything except the verified Razorpay webhook handler and the admin-triggered refund action — no other code path writes to this table, per TRD §8's audit-trail requirement.

10. Reviews / Testimonials

10.1

reviews

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
user_id	BIGINT UNSIGNED	FK → users.id, ON DELETE SET NULL, NULL (admin can also create curated testimonials without a real user account — matches current static testimonials on tytluxe.in)
vertical	ENUM('hotel','cruise','staycation','package','general')	NOT NULL, DEFAULT 'general'
reference_id	BIGINT UNSIGNED	NULL (polymorphic-lite, §12.1)
author_name	VARCHAR(191)	NOT NULL (denormalized — allows manual testimonials without a user row)
author_location	VARCHAR(191)	NULL (e.g., "Jaipur", matches current site's testimonial format)
avatar_path	VARCHAR(255)	NULL
rating	TINYINT UNSIGNED	NULL, CHECK (1-5)
body	TEXT	NOT NULL
is_published	BOOLEAN	NOT NULL, DEFAULT false (admin moderation gate, per PRD §8's deferred public-review-system caution — even curated testimonials get an approval step)

Indexes: INDEX(vertical, reference_id), INDEX(is_published).

11. Supporting / System Tables

11.1 `newsletter_subscribers`

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
email	VARCHAR(191)	UNIQUE, NOT NULL
subscribed_at	TIMESTAMP	NOT NULL
unsubscribed_at	TIMESTAMP	NULL

11.2 `admin_activity_logs`

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
actor_id	BIGINT UNSIGNED	FK → users.id, ON DELETE SET NULL, NULL
action	VARCHAR(100)	NOT NULL (e.g., "booking.status_override", "hotel.deactivated", "payment.refund_triggered")
subject_type	VARCHAR(100)	NOT NULL (model class name)
subject_id	BIGINT UNSIGNED	NOT NULL
meta	JSON	NULL (before/after values, reason text)

Indexes: INDEX(actor_id), INDEX(subject_type, subject_id), INDEX(created_at).

Required for: every manual booking status override, refund trigger, and role change per App Flow §15.5 and TRD §8's audit requirements — write via a model observer/event listener, not scattered manual calls, so nothing is missed.

11.3 `settings`

Simple key-value table for admin-editable site settings (App Flow §15.8):

Column	Type	Constraints
id	BIGINT UNSIGNED	PK
key	VARCHAR(100)	UNIQUE, NOT NULL
value	TEXT	NULL

Seeded keys: `contact_phone`, `contact_whatsapp`, `office_address`, `default_agent_id`, `social_links` (JSON-encoded in value), `email_notifications_enabled`.

11.4 Laravel Framework Tables (standard, not custom-designed but listed for completeness)

Table	Purpose
<code>sessions</code>	DB-driven session storage (TRD §3 — chosen over file sessions for multi-server readiness); columns: <code>id</code> , <code>user_id</code> (nullable FK), <code>ip_address</code> , <code>user_agent</code> , <code>payload</code> , <code>last_activity</code>
<code>password_reset_tokens</code>	<code>email</code> , <code>token</code> , <code>created_at</code> — used by §12.4 Forgot/Reset Password flow
<code>jobs</code> , <code>failed_jobs</code>	Laravel queue tables (TripJack polling, notification sending, per TRD §3)
<code>cache</code> , <code>cache_locks</code>	If using DB cache driver in any environment without Redis
<code>notifications</code>	Laravel's polymorphic notifications table, if using DB-channel notifications for in-app admin alerts (App Flow §15.4's notification badge)

12. Cross-Cutting Schema Patterns

12.1 "Polymorphic-lite" reference pattern (`vertical` + `reference_id`)

Used in `enquiries`, `offers`, `reviews`. Rather than Laravel's full `morphTo` (which needs a `reference_type` string column), this schema uses an explicit `vertical` ENUM + plain `reference_id` BIGINT, resolved in application code (a match/switch on `vertical` to determine which model to query). **Reason:** the set of possible verticals is small, fixed, and already needs its own enum for filtering/reporting (App Flow §15.4's filter-by-vertical requirement) — an ENUM is more indexable and query-filterable than a `morph_type` string column, and avoids accidentally polymorphic-joining against unrelated tables. Trade-off: referential integrity for `reference_id` is enforced in application code (Form Requests + model observers validating the reference exists for the given vertical before save), not by a DB foreign key, since a single column can't FK to five different tables. This must be covered by tests, not assumed safe.

12.2 Soft deletes vs. hard deletes

Soft-deleted: `users`, `hotels`, `cruises`, `staycations`, `packages` — anything a `booking` or `enquiry` might historically reference; deleting the listing must never break the ability to view a past booking's details. Hard-deleted (cascade): `hotel_images`, `room_types`, pivot tables, `cruise_itinerary_days`, etc. — child content with no independent historical meaning once its parent is gone. Never deleted, ever (append-only): `payments`, `admin_activity_logs` — financial and audit records are immutable history.

12.3 Money & currency

All monetary columns (`DECIMAL(10,2)`), paired with a (`currency CHAR(3)`) column wherever money is stored standalone (bookings, payments) — even though INR is the only currency at launch, this avoids a breaking schema change if international currency support is ever added, per TRD §8's forward-compatible posture on similar deferred features.

13. Authentication & Session Handling

13.1 Customer authentication

- **Mechanism:** Laravel Breeze (Blade edition), session-based (cookie + server-side session in the `sessions` table), per TRD §5.
- **Guard:** `web` guard, default Laravel `users` table, `role = customer` (agents/admins also live in `users` but are routed to a separate guard for panel access — see §13.2).
- **Session lifetime:** standard Laravel config (e.g., 120 minutes idle timeout), extendable via "Remember me" (`remember_token`), long-lived cookie).
- **Guest sessions:** guests completing bookings/enquiries are **not** authenticated but their in-progress search/booking state (destination, dates, selected room, etc.) is held in the Laravel session (not a DB row) until the booking is created — at booking creation, `guest_email`/`guest_phone` on the `bookings`/`enquiries` row become the durable record, not the session.
- **Session security:** `session` cookies `httponly`, `secure` (HTTPS-only per TRD §8), `samesite=lax`. CSRF token per Laravel default on all state-changing requests.

13.2 Admin/Agent authentication (Filament)

- **Separate guard:** `admin` guard (or a Filament panel scoped to `role IN ('agent', 'admin')`) on the same `users` table — either approach is acceptable; a fully separate `admins` table was considered and rejected because agents/admins don't need a parallel identity system, just elevated permissions on the same account model, and keeping one `users` table simplifies the `assigned_agent_id` FK in `enquiries` referencing the same table booking customers use).
- **Panel access rule:** Filament panel provider restricts login to `role IN ('agent', 'admin')` AND `is_active = true` — a `customer`-role user attempting `/admin` login is rejected with a generic "Invalid credentials" (never reveal that the account exists but lacks panel access, to avoid information leakage).
- **2FA:** required for `role = admin`, strongly recommended (configurable to required) for `role = agent`, per TRD §5 — implemented via Filament's built-in 2FA plugin or Fortify, backed by `two_factor_secret`/`two_factor_recovery_codes` on `users`.
- **Session:** same `sessions` table, same cookie security rules as §13.1, but with a shorter idle timeout recommended for the admin guard (e.g., 30–60 min) given it handles PII/payment data.

13.3 Password reset

Standard Laravel flow via `password_reset_tokens` table (§11.4) — works identically for both customer and admin/agent accounts since they share `users`.

14. Permissions & Roles

14.1 Role definitions

Role	Can access public site as	Can access /admin	Scope
customer	Logged-in customer	No	Own bookings/enquiries/profile only
agent	N/A (agents don't need the public-facing account area)	Yes	Enquiries/bookings assigned to them (configurable — see §14.2) + all catalog content (read/limited write, per §14.3)
admin	N/A	Yes	Full access to everything

14.2 Filament resource policy matrix

Resource	Customer	Agent	Admin
Hotels/Cruises/Staycations/Packages (catalog)	— (public read via frontend, not Filament)	Read + Edit (not Delete)	Full CRUD
Offers	—	Read only	Full CRUD
Enquiries	—	Read/Update status + notes on assigned enquiries only (configurable to all enquiries if the business prefers a shared queue model — flag for stakeholder decision)	Full access, can reassign any enquiry
Bookings	—	Read only, no status override	Read + manual status override (audit-logged, §11.2) + refund trigger
Payments	—	Read only (no PII beyond what's needed — consider masking full Razorpay raw_response from agent view)	Full read (refunds happen via Bookings resource action, not direct edits here)
Users (customers)	—	Read only (name/email/phone, for enquiry context)	Full read; can deactivate
Users (agent/admin accounts)	—	—	Full CRUD (role assignment)
Settings	—	—	Full access
Admin Activity Logs	—	—	Read only (even admins)

Resource	Customer	Agent	Admin
			don't edit audit logs)

Implementation: Filament Policies (Laravel's standard `Policy` classes, one per model) enforce this table — e.g., `BookingPolicy::update()` returns `true` only for `role === 'admin'`, `BookingPolicy::view()` returns `true` for `role IN ('agent', 'admin')`. This is standard Laravel authorization, invoked automatically by Filament resources — no custom permission engine needed given only 3 roles.

14.3 Catalog write restrictions for agents

Agents can edit existing hotel/cruise/staycation/package listings (e.g., fix a typo, update price) but cannot **delete** or **deactivate** them, and cannot create brand-new listings — that requires `admin`, since taking a whole product off the live site is a higher-stakes action than editing content. (Flag for stakeholder confirmation — this is a reasonable default, not a stated requirement from the PRD, so confirm before building the policy as a hard rule.)

15. Data Ownership Rules

These rules govern **row-level access** on top of the role-based Filament policies in §14 — i.e., not just "can an agent see the Bookings resource," but "can this specific agent see this specific booking."

- Customer data ownership:** A logged-in customer can only ever query/view `bookings`, `enquiries`, `payments` (via booking) where `bookings.user_id = auth()->id()` or `enquiries.user_id = auth()->id()`. Enforced via Eloquent global scopes on the customer-facing (`web` guard) controllers — never trust a route parameter (`/account/bookings/{id}`) without also checking ownership; a customer requesting another customer's booking ID gets a 404 (App Flow §14.1), not a 403 — never confirm the record exists to an unauthorized viewer.
- Guest data ownership:** A guest (no account) can only access a specific booking's confirmation page via a **signed URL** (Laravel's signed route feature — the `/booking/{ref}/confirmation` link emailed to them, cryptographically signed and optionally time-limited) — never via a guessable sequential ID alone.
- Agent enquiry/booking visibility:** Per §14.2's flagged decision — default recommendation is agents see only enquiries where `assigned_agent_id = auth()->id()` (plus all **unassigned** `status='new'` enquiries, so nothing sits invisible to everyone) OR the business may prefer agents see a fully shared queue. **This must be confirmed before building** — it changes the query scope materially.
- Admin visibility:** unrestricted, but every write action by an admin that touches booking status, refunds, or user roles must write to `admin_activity_logs` (§11.2) — visibility without accountability is not acceptable for financial data, per TRD §8.
- PII minimization:** `booking_travelers.passport_number` is encrypted at rest (Laravel's `encrypted` cast) and never included in any list view or export by default — only visible

on the single booking detail view to `admin` role, and to `agent` only if explicitly required for airline-facing tasks (flag for confirmation; default to admin-only if unsure).

- Data retention:** enquiries/bookings are retained indefinitely by default (business/financial record-keeping), but `deleted_at` (soft delete) should be used if a customer requests data deletion (e.g., under applicable privacy law) — hard-deleting a `user` row cascades to `SET NULL` on `bookings.user_id`/`enquiries.user_id` (per FK rules above) rather than deleting the booking/enquiry itself, preserving business records while removing the personal identity link. Flag with legal/compliance before finalizing exact retention periods.

16. Indexing Summary (performance-critical queries)

Query pattern	Supporting index
Hotel search by destination + category + active	<code>hotels(destination_id, category, is_active)</code> composite, or separate single-column indexes (composite preferred if this exact filter combo is the dominant query)
Admin enquiry list filtered by status + vertical, sorted by date	<code>enquiries(status, vertical, created_at)</code>
Admin booking list filtered by status, sorted by date	<code>bookings(status, created_at)</code>
Customer's own bookings	<code>bookings(user_id)</code>
Guest booking lookup by email	<code>bookings(guest_email)</code>
Offer validity window queries (scheduled job + public Offers page)	<code>offers(valid_from, valid_to, is_active)</code>
Destination/hotel/cruise/package autocomplete search	<code>FULLTEXT</code> indexes per §2.2, §3.1, §4.1, §5.1, §6.1
Payment lookup by Razorpay IDs (webhook processing)	<code>UNIQUE</code> on <code>razorpay_order_id</code> , <code>razorpay_payment_id</code> (already required for idempotency, doubles as the lookup index)

17. Open Schema Questions for Stakeholder/Engineering Input

- Confirm agent enquiry/booking visibility model (assigned-only vs. shared queue) — §14.2/§15.3 — this is a meaningful query-scope decision, not a detail to leave implicit.
- Confirm whether `booking_travelers.passport_number` should ever be visible to agents (§15.5) or strictly admin-only.

- Confirm data retention/deletion policy with legal input before finalizing §15.6 as a hard rule rather than a placeholder.
 - Decide now whether to build the `booking_flight_legs` child table (§9.1 note) up front for multi-city flights, or defer until TripJack's actual multi-city response shape is confirmed (App Flow §17 already flagged this as unresolved).
 - Confirm whether catalog listing creation/deletion should really be admin-only (§14.3) or if agents should also create new listings — affects the Filament policy class before it's written.
-

This schema is the implementation source of truth for migrations and Eloquent models. Where this document and the TRD's illustrative schema (§4.2) differ in small ways, this document supersedes it as the more detailed, later-authored version — reconcile the TRD's summary table to match this before development begins.